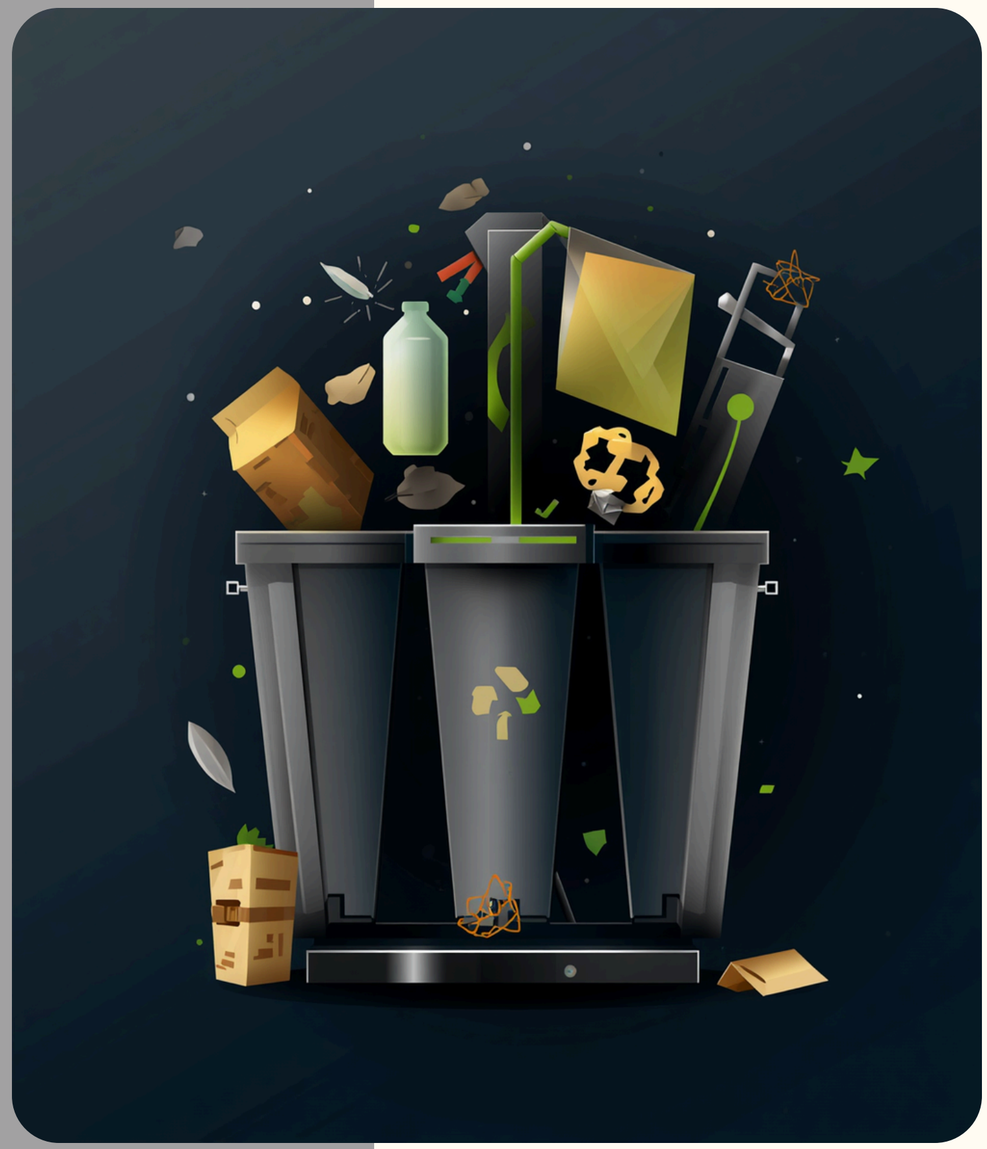


The Sortitude

Real time trash sorting with edge AI

Team Member:

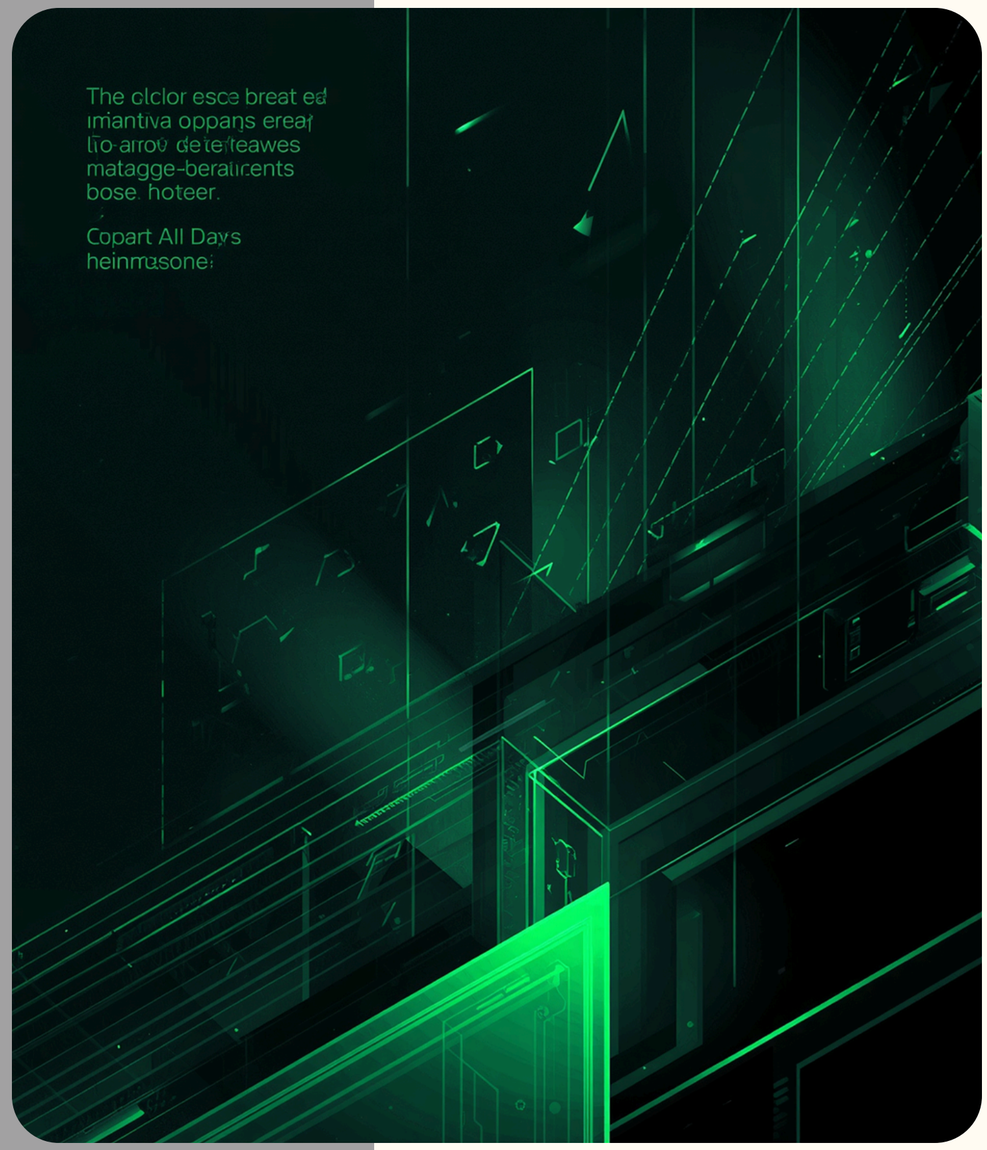
Abhi Faldu
Aatman Sabhaya
Jeel Sidpara



The Problem

Why Automated Waste Sorting Matters

- Recycling fails when materials are mixed.
- Contaminated recycling can send whole batches to landfill.
- Manual sorting is slow, costly, and inconsistent
- A low power camera at the bin could classify waste directly at the source.
- The solution should run locally on device without cloud, low power and in real time.



Goals & Success Criteria

What we set out to do

- Train an image classifier for 6 waste types: cardboard, glass, metal, paper, plastic, trash
- Compress the model with quantization and pruning
- Prepare deployment for the Google Coral Dev Board
- Compare model variants by size, accuracy, and deployment readiness
- Run a live camera demo or fallback CPU demo



Google Coral Dev Board

The edge device used in our project

- Integrated Edge TPU coprocessor for ML inference.
- Around 4 TOPS performance at low power.
- NXP i.MX 8M SoC, quad Cortex A53.
- 1–4 GB LPDDR4 RAM, 8 GB eMMC
- Runs on Mendel Linux.
- Managed through USB, SSH and MDT.



System Pipeline

End to end Waste Classification Flow

TrashNet → MobileNetV2 → TFLite Conversion → Quantization / Pruning → Edge TPU Compile → Coral Deployment → Demo

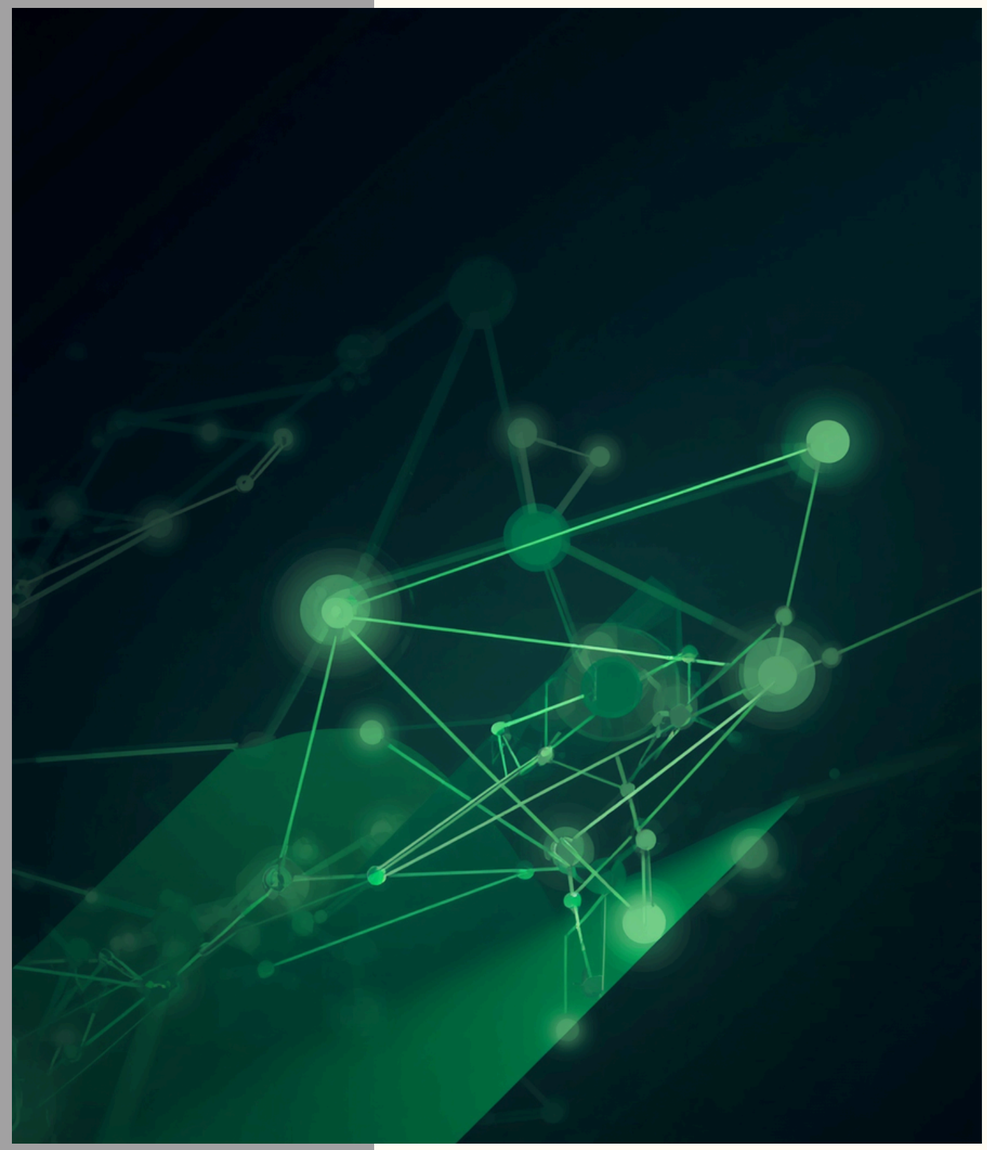
- Training and compression: Google Colab
- Host setup: WSL2 + MDT + USB passthrough
- Deployment files: model, labels.txt, demo.py
- Final Coral deployment: prepared, but blocked by board access issue

Dataset: TrashNet

Waste image dataset

- Around 2,500 labeled images used.
- 6 waste classes were created cardboard, glass, metal, paper, plastic and trash.
- Images resized to 224 × 224 px.
- 80/20 train validation split.
- Suitable for transfer learning because the dataset is relatively small.





MobileNetV2 Transfer Learning

Why this model?

- MobileNetV2 is lightweight and suitable for mobile/edge inference
- It is faster and smaller than many standard CNN architectures
- It works well with transfer learning
- It is quantization-friendly for TFLite deployment

Training approach

- Used ImageNet-pretrained MobileNetV2
- Replaced final layer with a 6-class classifier
- Phase 1: freeze base model and train classifier head
- Phase 2: fine-tune top layers with low learning rate

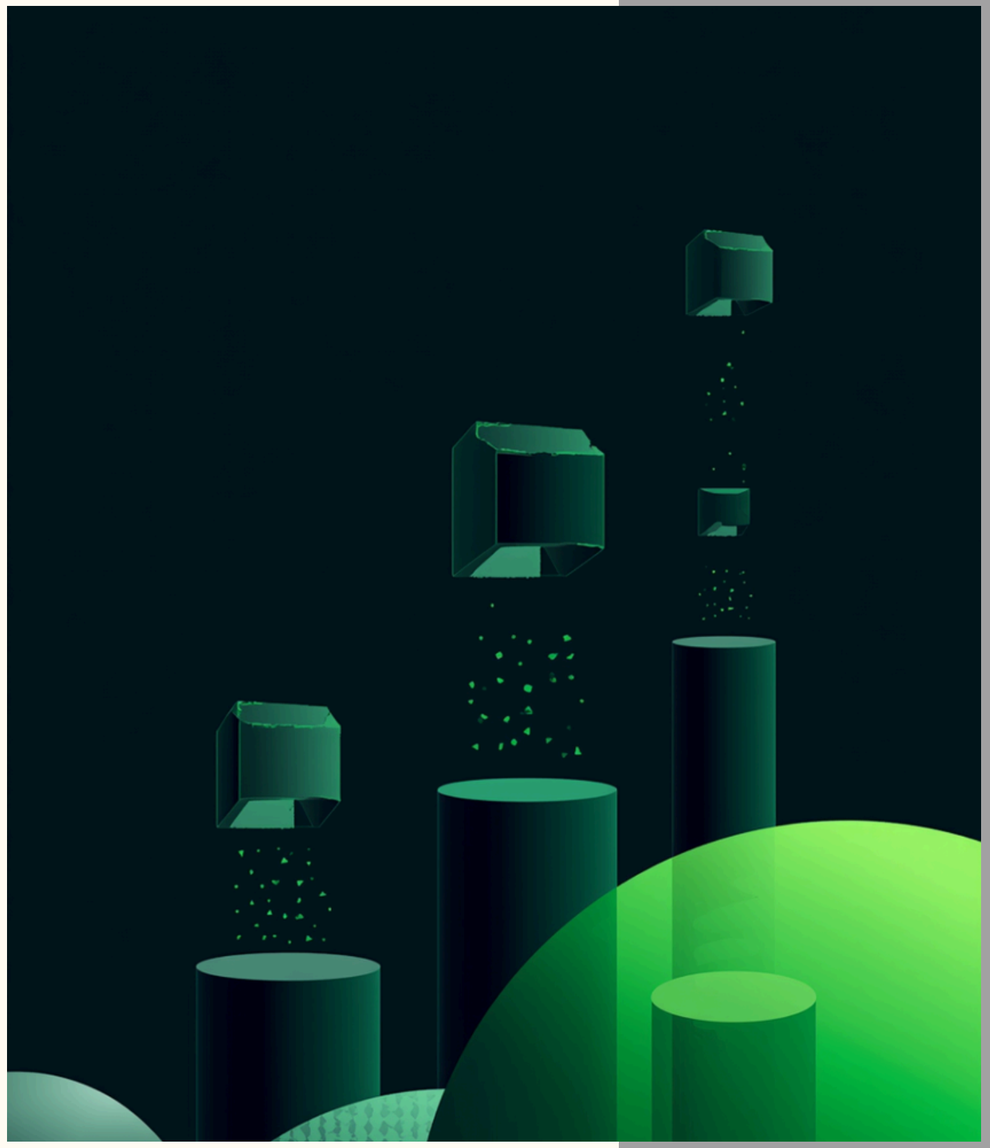
Baseline validation accuracy: **92.87%**

Model Compression: 3 Variants

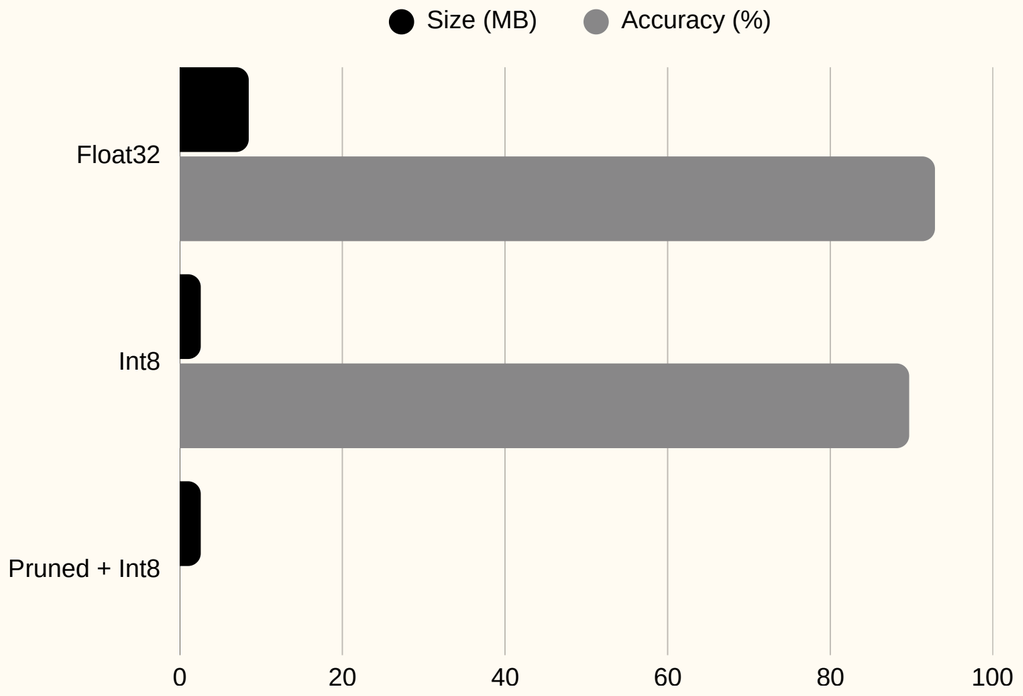
Trading size for deployability

Variant	What it does	Deployment meaning
Float32 baseline	Full precision model	Accurate, but not Edge-TPU accelerated
Int8 quantized	Converts weights and activations to 8-bit integers	Edge-TPU compatible
Pruned + Int8	Removes less important weights, then quantizes	Smaller in theory, failed in practice

Quantization was necessary because the Edge TPU requires int8 TFLite models.



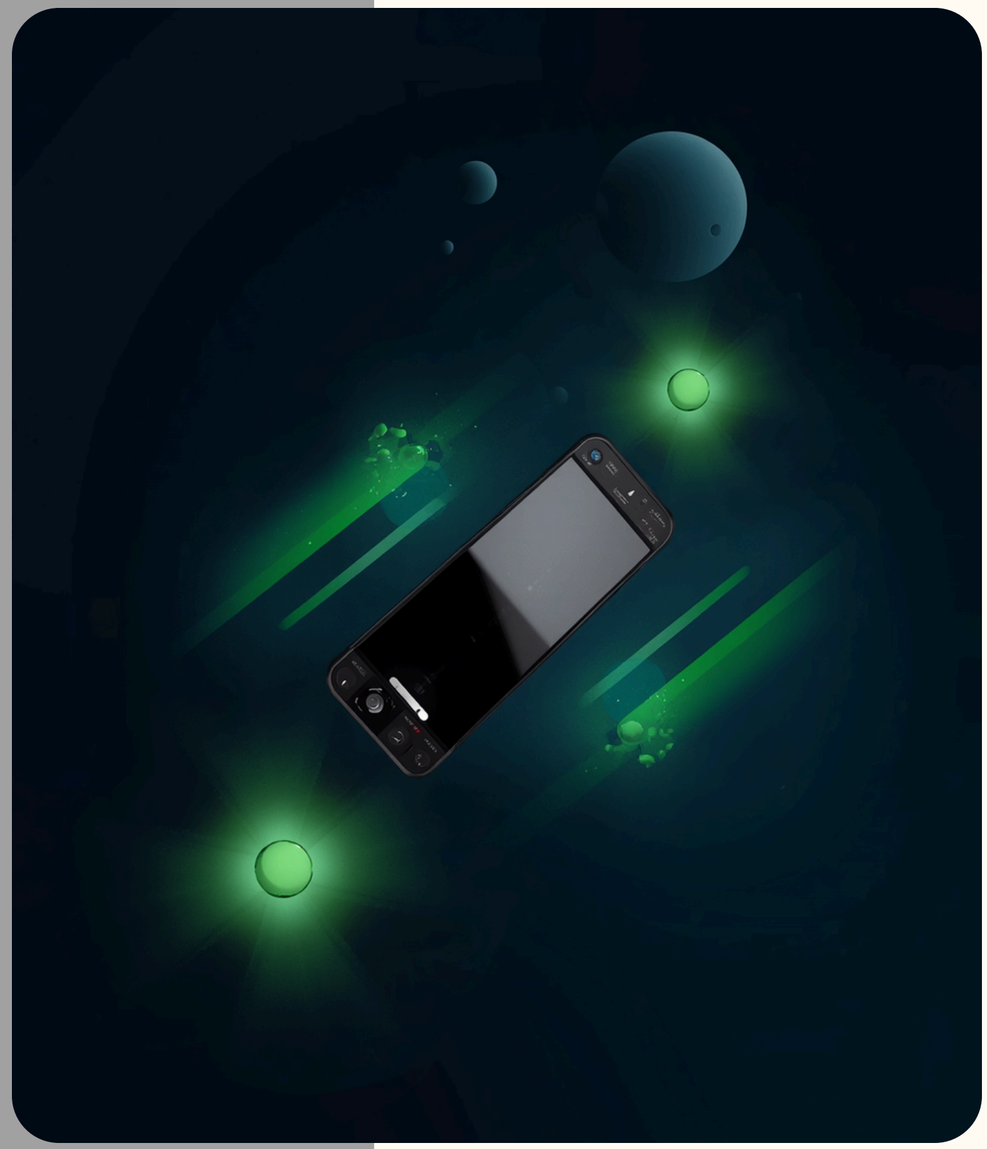
Size vs. Accuracy



Results

The int8 model is 3.3× smaller than the float32 baseline with only a 3.17 percentage-point accuracy drop.

Model	Size	Accuracy	Result
Float32 baseline	8.49 MB	92.87%	Accurate, CPU only
Int8 quantized	2.59 MB	89.70%	Best deployable model
Pruned 50% + Int8	2.59 MB	0%	Failed after compression



Deployment Plan

Coral board setup process

- Flash Mendel Linux to the Coral Dev Board.
- Connect the board through USB/SSH connections.
- Pair host and board using MDT.
- Transfer model, labels and demo script for analysis.
- Run live camera inference with demo.py.



Problems & Challenges

Where it got difficult

1. Board access: The SSH key was configured locally and was not shared or backed up. Because of this, we could not access the board through MDT/SSH or push model files.

2**. Label export bug:** The first labels.txt file contained an incorrect string instead of the actual class names.

3**. USB passthrough:** USB forwarding through WSL2 and usbipd-win required extra troubleshooting.



Thank You

Team

Abhi Faldu · Aatman Sabhaya · Jeel Sidpara

Project

The Sortitude — Real time trash sorting with edge AI